



AI Security

- Lecture 03. Decision Trees & Ensemble Models

Artificial Intelligence Research (AIR) LAB

<https://air.korea.ac.kr/>

School of Cybersecurity

Korea University

Sangkyun Lee (이상근)

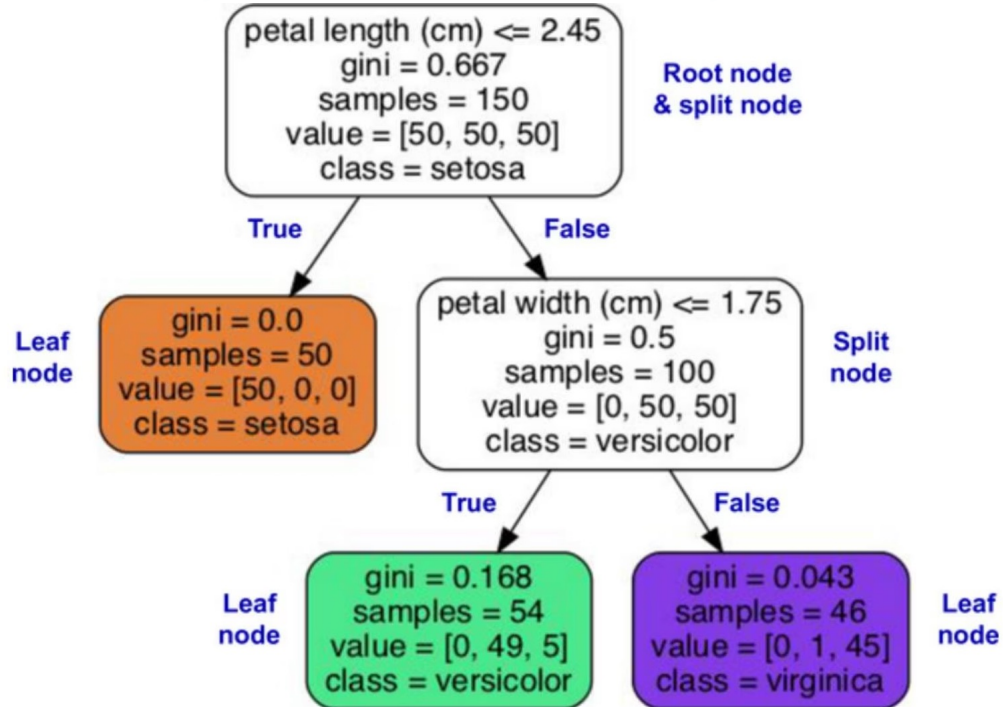
2025. Spring Semester

결정 트리

결정 트리는 분류와 회귀 작업을 모두 수행할 수 있는 다재다능한 머신러닝 알고리즘임. 복잡한 데이터셋에 적합한 강력한 알고리즘으로, 캘리포니아 주택 데이터셋에 완벽하게 적합(사실은 과적합)시킬 수 있음.

결정 트리는 또한 랜덤 포레스트의 기본 구성 요소이며, 이는 오늘날 가장 강력한 머신러닝 알고리즘 중 하나임. 이 장에서는 결정 트리의 훈련, 시각화, 예측 방법을 살펴본 후 Scikit-Learn에서 사용하는 CART 훈련 알고리즘을 살펴보고 트리 정규화 방법과 회귀 작업에 사용하는 방법을 탐색함.

결정 트리 훈련 및 시각화



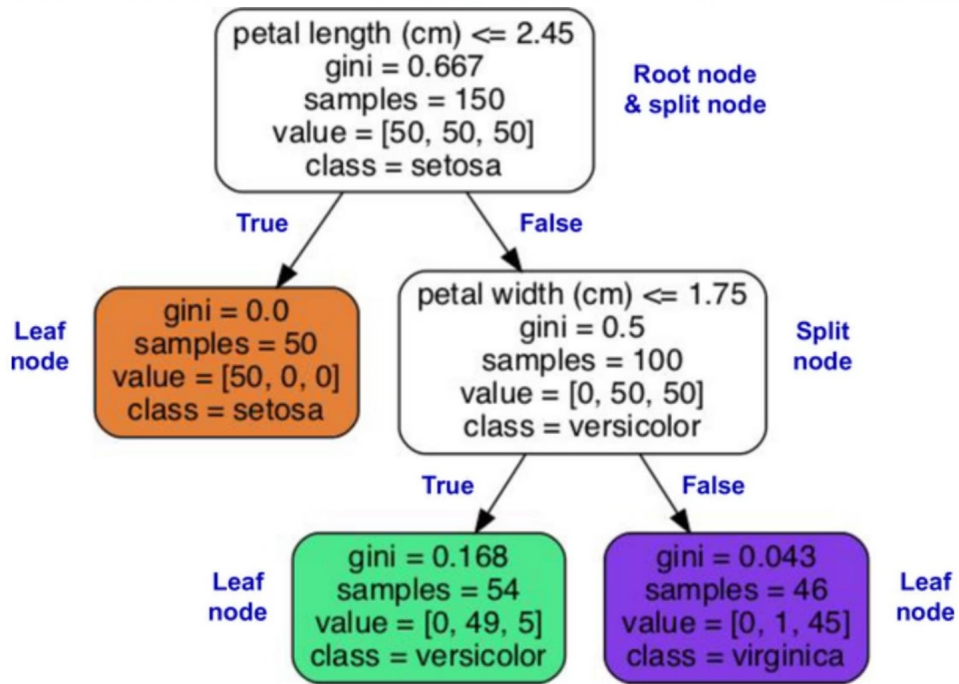
결정 트리를 이해하기 위해 하나를 구축하고 예측 방법을 살펴보자. 아이리스 데이터셋에 `DecisionTreeClassifier`를 훈련시키는 과정을 통해 결정 트리의 작동 원리를 파악할 수 있음.

트리 훈련

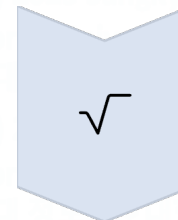
Scikit-Learn의 `DecisionTreeClassifier`를 사용하여 아이리스 (IRIS) 데이터셋에 대해 `max_depth=2`로 설정하여 모델을 훈련시킴.

결정 트리는 직관적이고 해석하기 쉬운 시각적 표현을 제공하여 모델의 의사 결정 과정을 명확하게 이해할 수 있게 함. 이러한 시각화를 통해 모델이 어떤 특성을 기준으로 데이터를 분류하는지 파악할 수 있음.

예측 방법

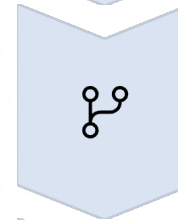


아이리스 꽃을 꽃잎을 기준으로 분류하는 경우:



루트 노드 (깊이 0)

꽃잎 길이가 2.45cm보다 작은지 확인함



자식 노드 (깊이 1)

조건에 따라 왼쪽(작음) 또는 오른쪽(큼) 노드로 이동함

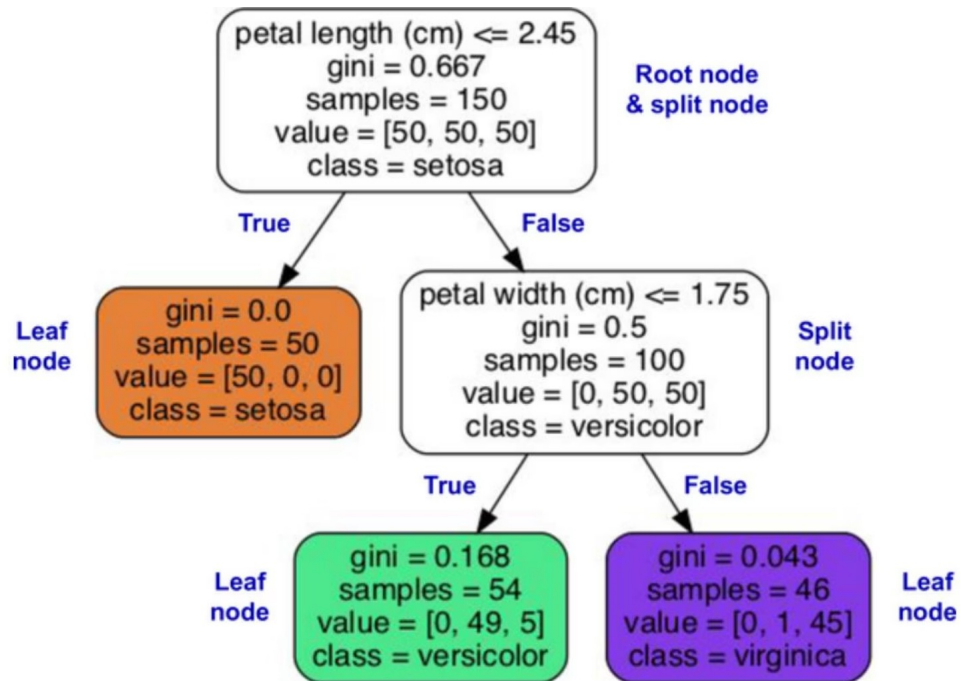


리프 노드 (깊이 2)

최종 예측 클래스 결정(Setosa, Versicolor, Virginica)

Scikit-Learn은 CART (Classification & Regression Tree) 알고리즘을 사용하여 항상 정확히 두 개의 자식을 가진 이진 트리만 생성함. 다른 알고리즘(예: ID3)은 둘 이상의 자식을 가진 노드를 생성할 수 있음.

예측 방법



노드 속성 이해

- **samples:** 해당 노드에 적용되는 훈련 인스턴스 수
- **value:** 각 클래스별 훈련 인스턴스 수
- **gini:** 지니 불순도 측정값 (0은 완전 순수)

지니 불순도 계산식:

$$G_i = 1 - \sum_{k=1}^n p_{i,k}^2$$

여기서 G_i 는 i 번째 노드의 지니 불순도이고, $p_{i,k}$ 는 i 번째 노드에서 클래스 k 인스턴스의 비율임.

모델 해석: 화이트 박스 vs 블랙 박스

화이트 박스 모델

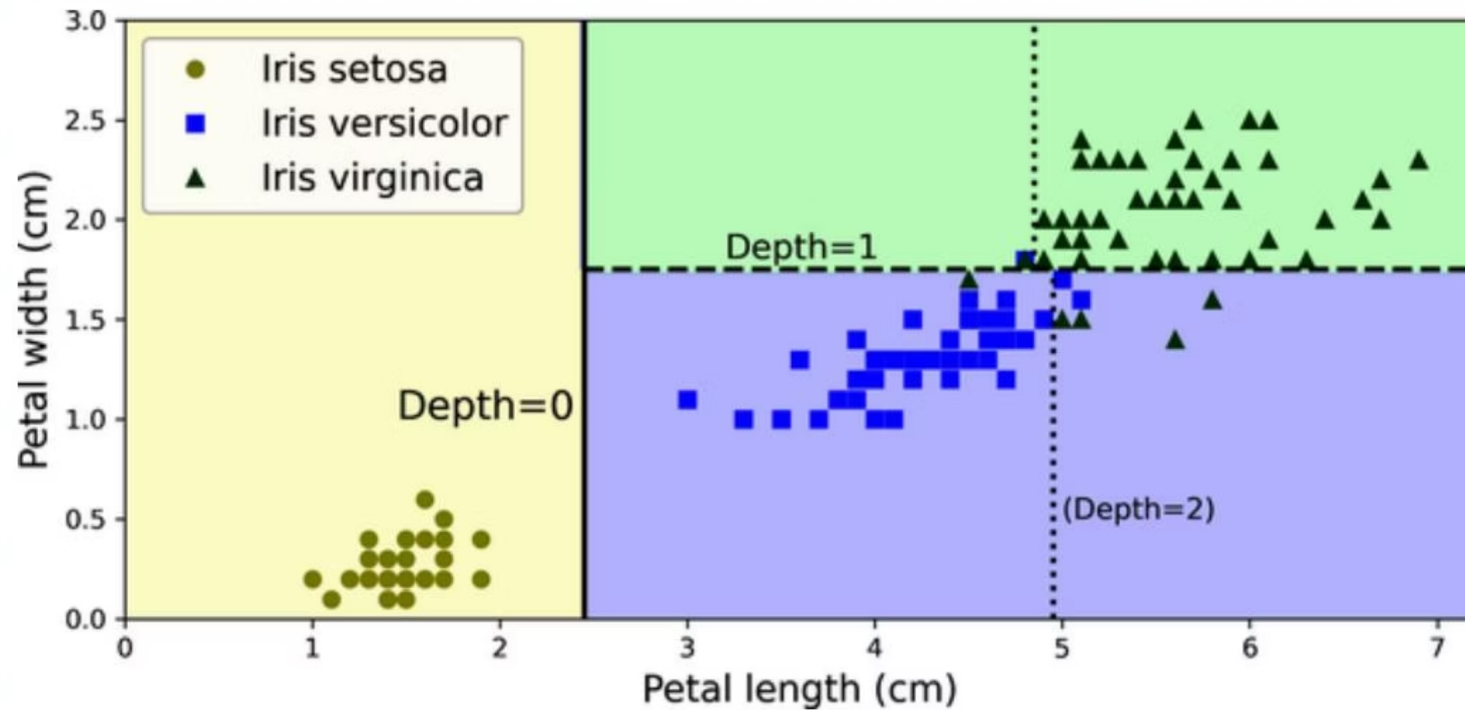
- 결정 트리는 직관적이고 해석하기 쉬움
- 의사 결정 과정이 명확하게 보임
- 필요한 경우 수동으로 적용 가능한 간단한 분류 규칙 제공
- 모델이 왜 특정 결정을 내렸는지 이해하기 쉬움

블랙 박스 모델

- 랜덤 포레스트와 신경망은 일반적으로 블랙 박스 모델로 간주됨
- 뛰어난 예측을 제공하지만 결정 과정 설명이 어려움
- 예: 신경망이 이미지에서 사람을 인식할 때 어떤 특징(눈, 입, 코 등)이 결정에 기여했는지 설명하기 어려움

해석 가능한 ML 분야는 인간이 이해할 수 있는 방식으로 결정을 설명할 수 있는 ML 시스템을 만드는 것을 목표로 함. 이는 시스템이 불공정한 결정을 내리지 않도록 하기 위해 많은 도메인에서 중요함.

클래스 확률 추정



트리 탐색

결정 트리는 인스턴스에 대한 리프 노드를 찾기 위해 트리를 탐색함

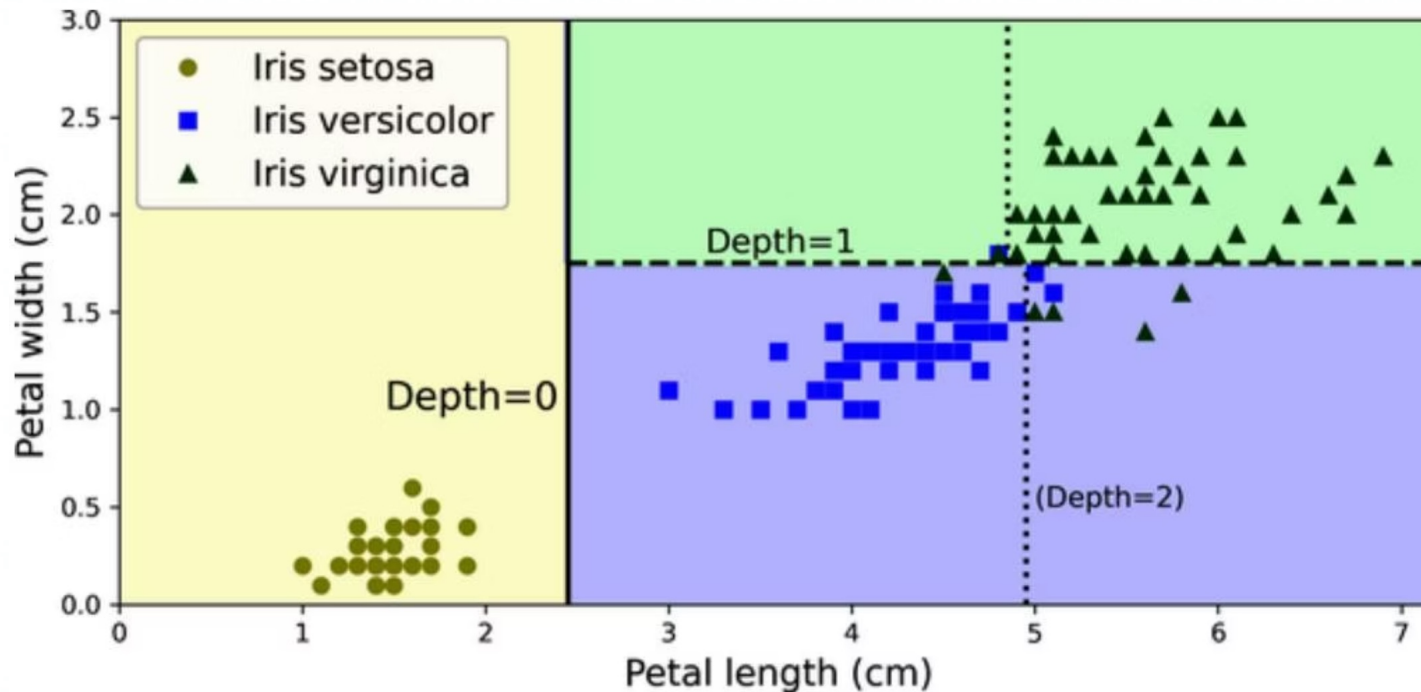
비율 계산

해당 노드에서 특정 클래스 k의 훈련 인스턴스 비율을 반환함

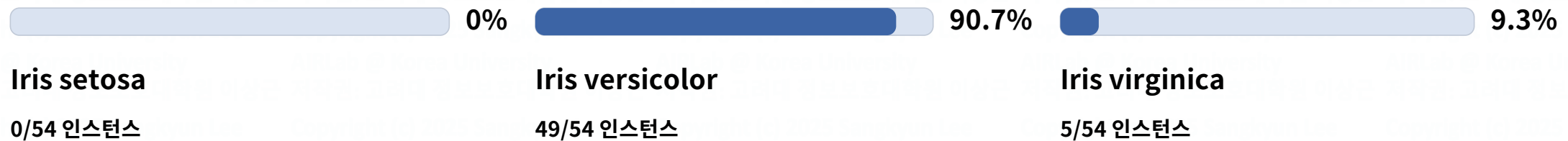
확률 출력

각 클래스에 대한 확률을 백분율로 제공함

클래스 확률 추정



예시: 꽃잎 길이가 5cm, 너비가 1.5cm인 새로운 꽃 → 해당 리프 노드는 깊이-2 왼쪽 노드이므로 결정 트리는 다음과 같은 확률을 출력함:



가장 높은 확률을 가진 *Iris versicolor*(클래스 1)를 예측 결과로 출력함. 같은 영역 내 어디서든 동일한 확률이 추정된다는 점에 유의해야 함.

CART 훈련 알고리즘

Scikit-Learn은 결정 트리를 훈련시키기 위해 CART(Classification and Regression Tree) 알고리즘을 사용함. 이 알고리즘은 다음과 같이 작동함:

1 분할 기준

단일 특성 k 와 임계값 t 를 사용하여 훈련 세트를 두 개의 하위 집합으로 분할함. 크기에 따라 가중치가 부여된 가장 순수한 하위 집합을 생성하는 (k, t) 쌍을 찾음.

2 비용 함수

알고리즘이 최소화하려는 비용 함수:

$$J(k, t_k) = \frac{m_{\text{left}}}{m} G_{\text{left}} + \frac{m_{\text{right}}}{m} G_{\text{right}}$$

3 재귀적 분할

CART 알고리즘은 훈련 세트를 성공적으로 두 개로 분할한 후 동일한 논리로 하위 집합을 분할하고, 그 다음 하위-하위 집합 등을 재귀적으로 분할함.

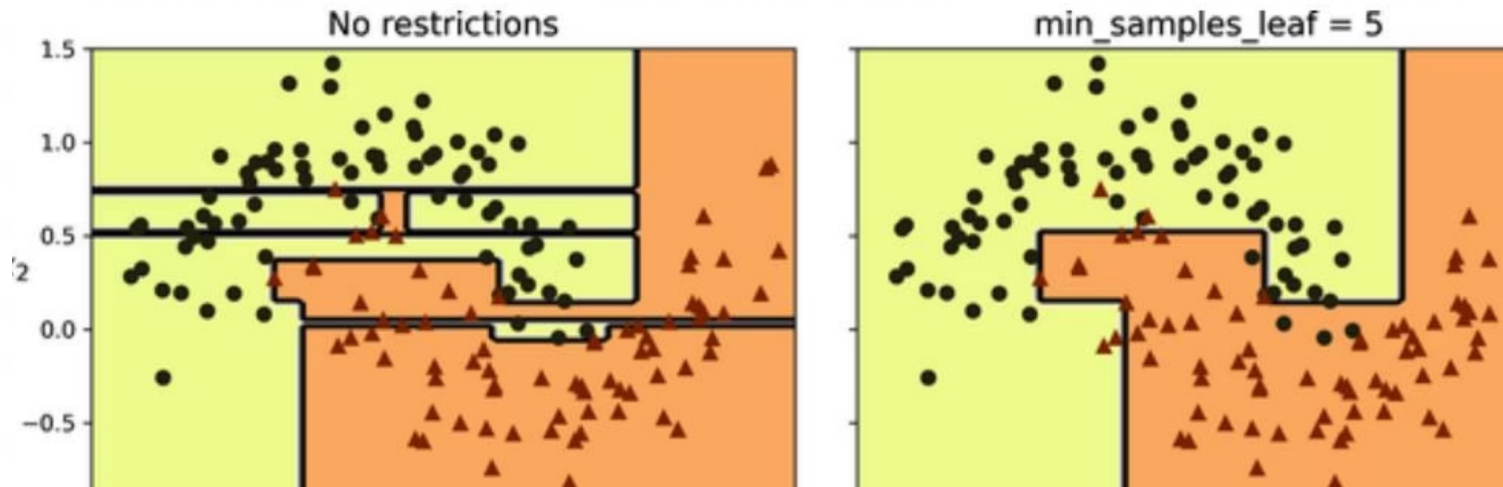
4 중단 조건

최대 깊이(max_depth)에 도달하거나 불순도를 줄일 수 있는 분할을 찾을 수 없을 때 재귀를 중단함. 추가 중단 조건:

min_samples_split, min_samples_leaf,
min_weight_fraction_leaf, max_leaf_nodes.

CART 알고리즘은 탐욕 알고리즘으로, 최상위 수준에서 최적의 분할을 탐욕적으로 검색한 다음 각 후속 수준에서 프로세스를 반복함. 여러 수준 아래에서 가능한 최저 불순도로 이어질지 확인하지 않음. 최적의 트리를 찾는 것은 NP-완전 문제로 알려져 있어 작은 훈련 세트에서도 문제가 다루기 어려움.

트리 깊이와 과적합 이슈 → 하이퍼파라미터 튜닝 필요



결정 트리의 깊이가 제한되지 않으면 트리 구조가 훈련 데이터에 매우 밀접하게 적합(과적합)됨.

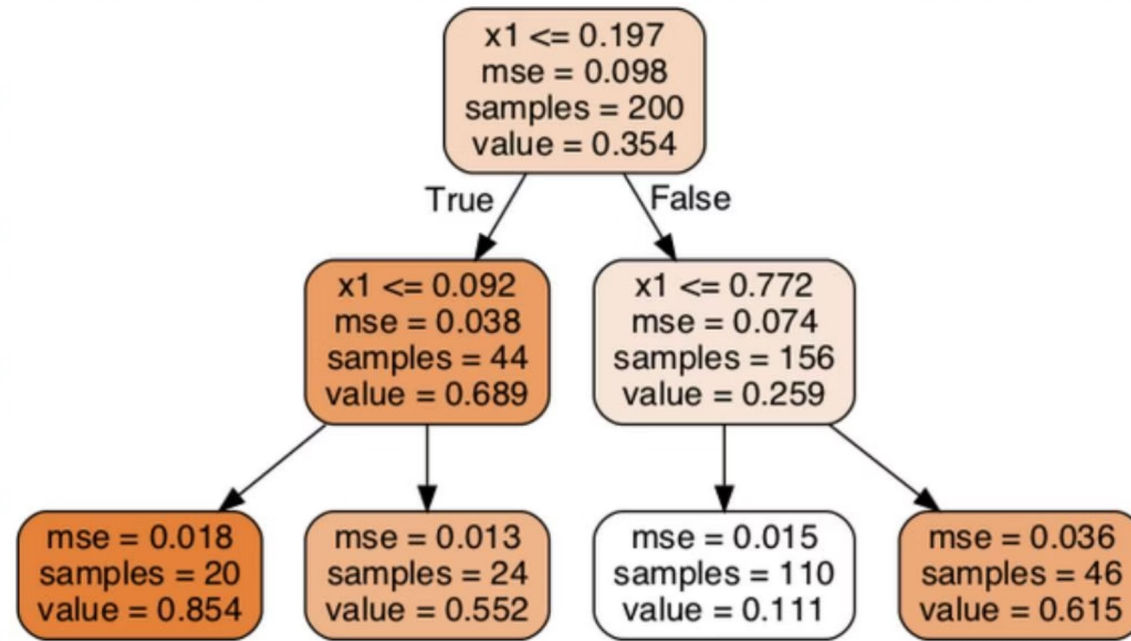
정규화 방법

과적합을 방지하기 위해 훈련 중 결정 트리의 자유도를 제한해야 함. 이를 정규화라고 함.

Scikit-Learn의 `DecisionTreeClassifier` 클래스는 결정 트리의 형태를 제한하는 여러 매개변수를 제공함:

- `max_depth`: 결정 트리의 최대 깊이
- `max_features`: 각 노드에서 분할을 위해 평가되는 최대 특성 수
- `max_leaf_nodes`: 최대 리프 노드 수
- `min_samples_split`: 노드가 분할되기 전에 가져야 하는 최소 샘플 수
- `min_samples_leaf`: 리프 노드가 생성되기 위해 가져야 하는 최소 샘플 수

회귀 트리



회귀 트리의 특징

회귀 트리는 분류 트리와 매우 유사하게 보이지만, 각 노드에서 클래스를 예측하는 대신 값을 예측함.

예를 들어, $x = 0.2$ 인 새 인스턴스에 대한 예측을 하려면:

1. 루트 노드: $x \leq 0.197$ 인지 확인 (아님)
2. 오른쪽 자식 노드로 이동: $x \leq 0.772$ 인지 확인 (맞음)
3. 왼쪽 자식 노드로 이동: 리프 노드이므로 value=0.111 예측

이 예측은 이 리프 노드와 연관된 110개 훈련 인스턴스의 평균 타겟 값임.

CART 알고리즘 (회귀용)

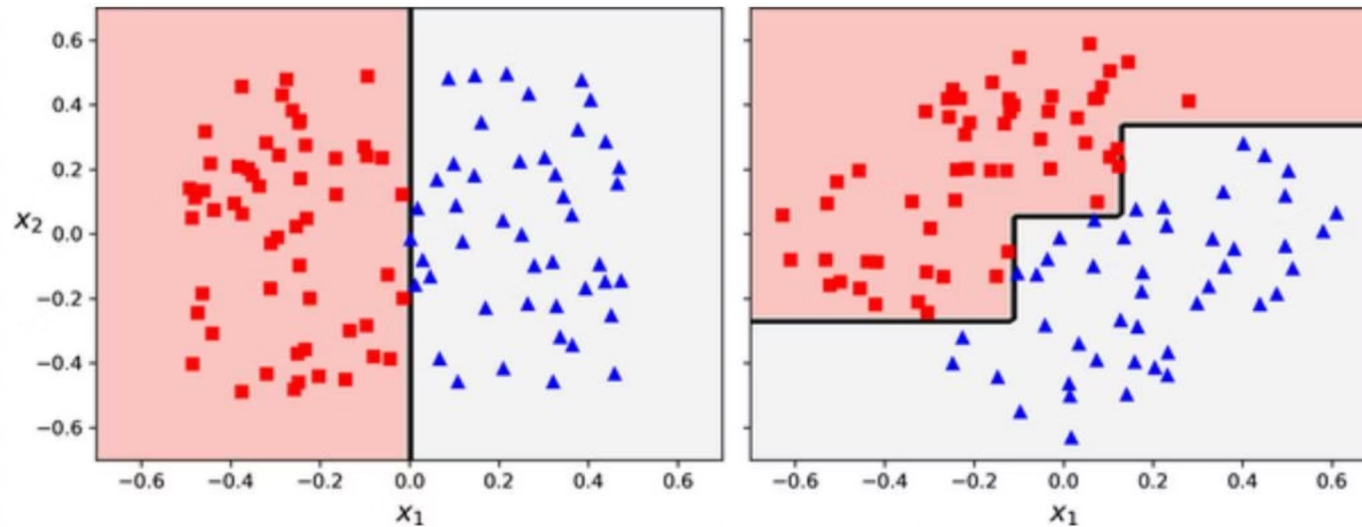
회귀를 위한 CART 알고리즘은 불순도를 최소화하는 대신 MSE(평균 제곱 오차)를 최소화하는 방식으로 훈련 세트를 분할하려고 함.

알고리즘이 최소화하려는 비용 함수:

$$J(k, t_k) = \frac{m_{\text{left}}}{m} \text{MSE}_{\text{left}} + \frac{m_{\text{right}}}{m} \text{MSE}_{\text{right}}$$

각 영역에 대한 예측값은 항상 해당 영역에 있는 인스턴스의 평균 타겟 값임. 알고리즘은 대부분의 훈련 인스턴스가 예측값에 최대한 가깝도록 각 영역을 분할함.

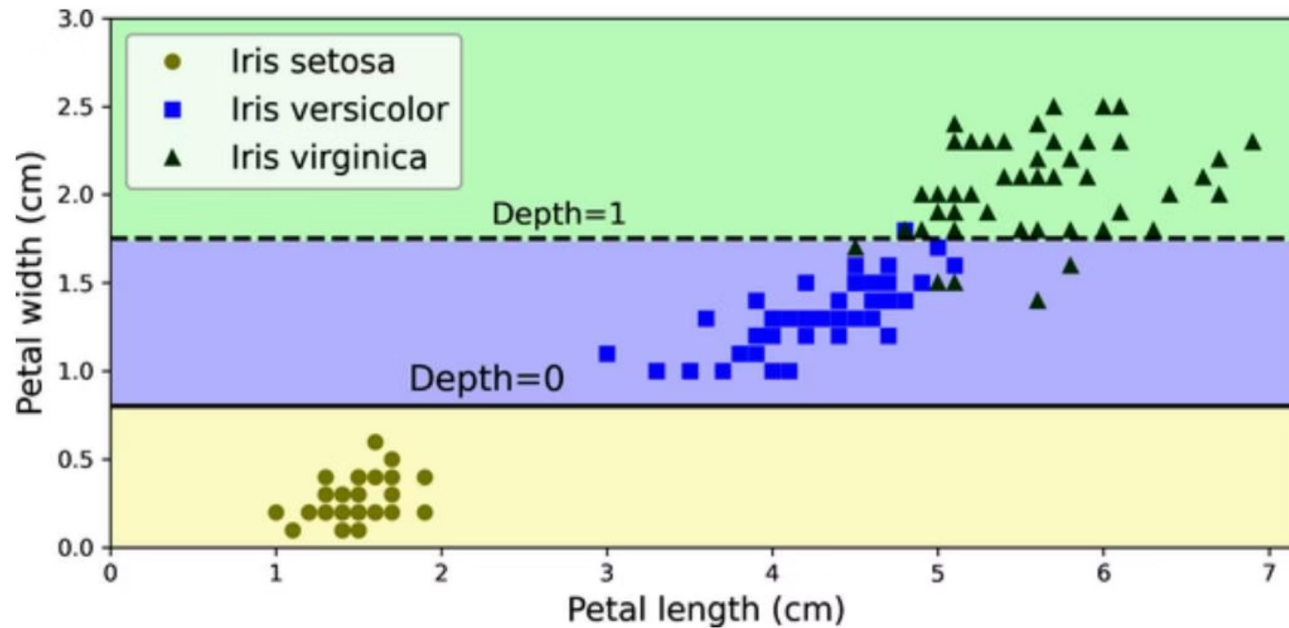
축 방향에 대한 민감성



방향 민감성 문제

간단한 선형 분리 가능 데이터셋에서 왼쪽의 결정 트리는 쉽게 분할할 수 있지만, 오른쪽에서는 데이터셋이 45° 회전된 후 결정 경계가 불필요하게 복잡해 보임. 두 결정 트리 모두 훈련 세트에 완벽하게 적합하지만, 오른쪽 모델은 일반화가 잘 되지 않을 가능성이 높음.

결정 트리의 높은 분산



분산 문제

결정 트리의 주요 문제는 상당히 높은 분산을 가진다는 것임. 하이퍼파라미터나 데이터의 작은 변화가 매우 다른 모델을 생성할 수 있음.

Scikit-Learn에서 사용하는 훈련 알고리즘은 확률적임 - 각 노드에서 평가할 특성 집합을 무작위로 선택함. 따라서 `random_state` 하이퍼파라미터를 설정하지 않으면 정확히 동일한 데이터에서 동일한 결정 트리를 재훈련해도 매우 다른 모델이 생성될 수 있음.

랜덤 포레스트 솔루션

다행히도 여러 트리에 걸쳐 예측을 평균화하면 분산을 크게 줄일 수 있음. 이러한 트리 앙상블을 랜덤 포레스트라고 하며, 오늘날 가장 강력한 모델 유형 중 하나임.

랜덤 포레스트는 여러 결정 트리의 예측을 결합하여 더 안정적이고 정확한 예측을 제공함. 각 트리는 훈련 데이터의 다른 부분집합에서 훈련되고 다른 특성 집합을 고려함.

이 방법은 개별 트리의 높은 분산 문제를 해결하면서도 결정 트리의 강력한 예측 능력을 유지함.

연습 문제

1 결정 트리 깊이

백만 개의 인스턴스가 있는 훈련 세트에서 (제한 없이) 훈련된 결정 트리의 대략적인 깊이는 얼마인가?

3 과적합 해결

결정 트리가 훈련 세트에 과적합된 경우, `max_depth`를 감소시키는 것이 좋은 아이디어인가?

2 지니 불순도 비교

노드의 지니 불순도는 일반적으로 부모 노드보다 낮은가 높은가?
일반적으로 낮거나 높은가, 아니면 항상 낮거나 높은가?

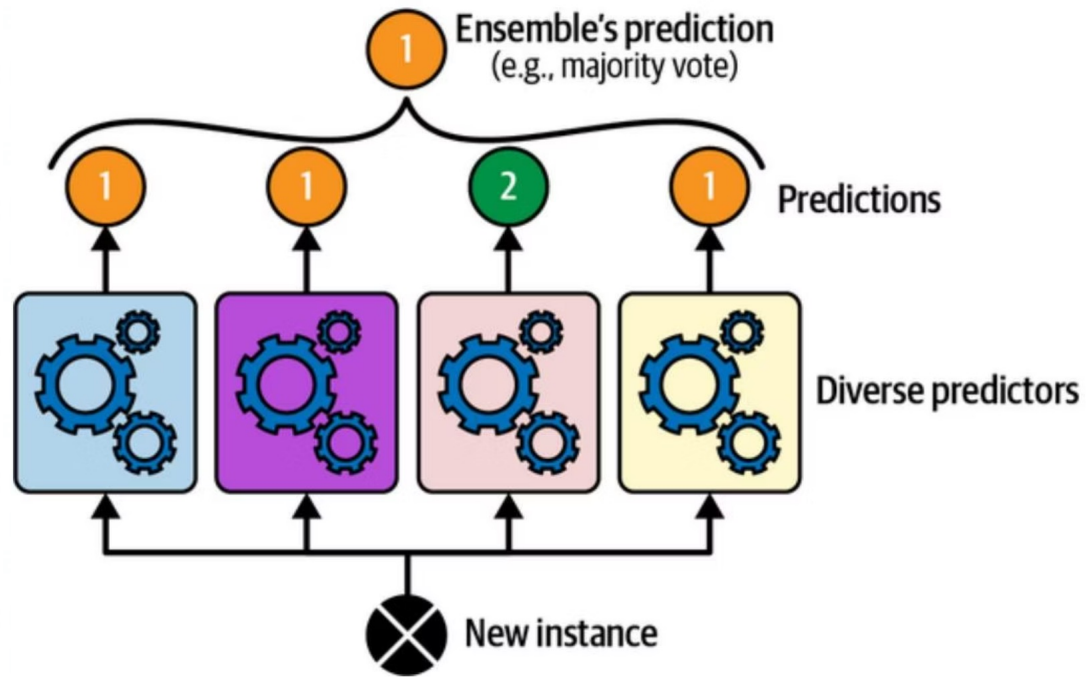
4 입력 특성 스케일링

결정 트리가 훈련 세트에 과소적합된 경우, 입력 특성을 스케일링하는 것이 좋은 아이디어인가?

앙상블 학습과 랜덤 포레스트

- 복잡한 질문을 수천 명의 사람들에게 물어보고 그 답변을 종합하면, 전문가의 답변보다 더 나은 결과를 얻을 수 있음. 이를 '군중의 지혜'라고 함. 마찬가지로, 여러 예측기(분류기나 회귀기)의 예측을 종합하면 단일 예측기보다 더 나은 결과를 얻을 수 있음.
- 예측기의 그룹을 하나의 예측기처럼 사용하는 것을 앙상블이라 함.
- 랜덤 포레스트는 결정 트리 분류기의 앙상블로, 단순함에도 불구하고 현재 가장 강력한 머신러닝 알고리즘 중 하나임.

투표 분류기



각각 약 80%의 정확도를 가진 여러 분류기를 훈련했다고 가정해 보자. 로지스틱 회귀, SVM, 랜덤 포레스트, k-최근접 이웃 분류기 등이 있을 수 있음.

(Hard Voting Classifier) 각 분류기의 예측을 집계하여 가장 많은 표를 받은 클래스를 앙상블의 예측으로 사용

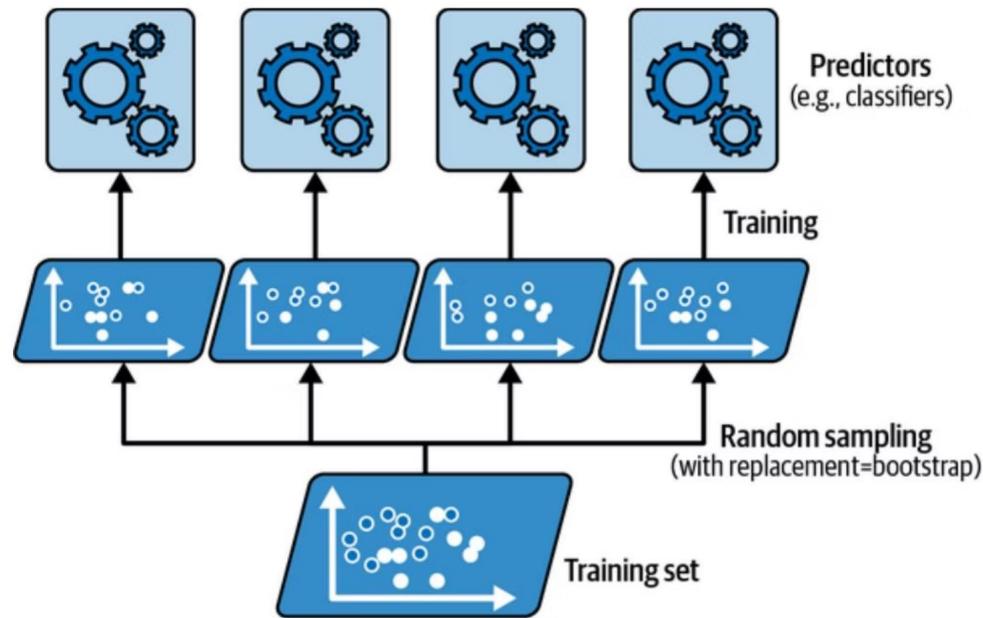
1 앙상블의 성능 향상

놀랍게도, 이 투표 분류기는 종종 앙상블 내 최고의 분류기보다 더 높은 정확도를 달성함. 각 분류기가 약한 학습기(무작위 추측보다 약간 나은 정도)라도, 충분한 수의 약한 학습기가 있고 충분히 다양하다면 앙상블은 강한 학습기(높은 정확도)가 될 수 있음.

2 다양성의 중요성

앙상블 방법은 예측기들이 서로 독립적일 때 가장 효과적임. 다양한 분류기를 얻는 한 가지 방법은 매우 다른 알고리즘을 사용하여 훈련하는 것임. 이는 매우 다른 유형의 오류를 만들 가능성을 높여 앙상블의 정확도를 향상시킴.

Bagging (Bootstrap Aggregation)과 Pasting



다양한 분류기 집합을 얻는 또 다른 방법은 동일한 훈련 알고리즘을 사용하되, 훈련 세트의 다른 무작위 부분 집합으로 훈련.

복원 추출로 샘플링할 때 이 방법을 배깅(부트스트랩 집계)이라 함. 비복원 추출로 샘플링할 때는 페이스팅이라 함.

목표

편향-분산

병렬

무작위 샘플링

각 예측기는 훈련 세트의 무작위 부분 집합에서 훈련됨. 이로 인해 예측기 간의 다양성이 증가함.

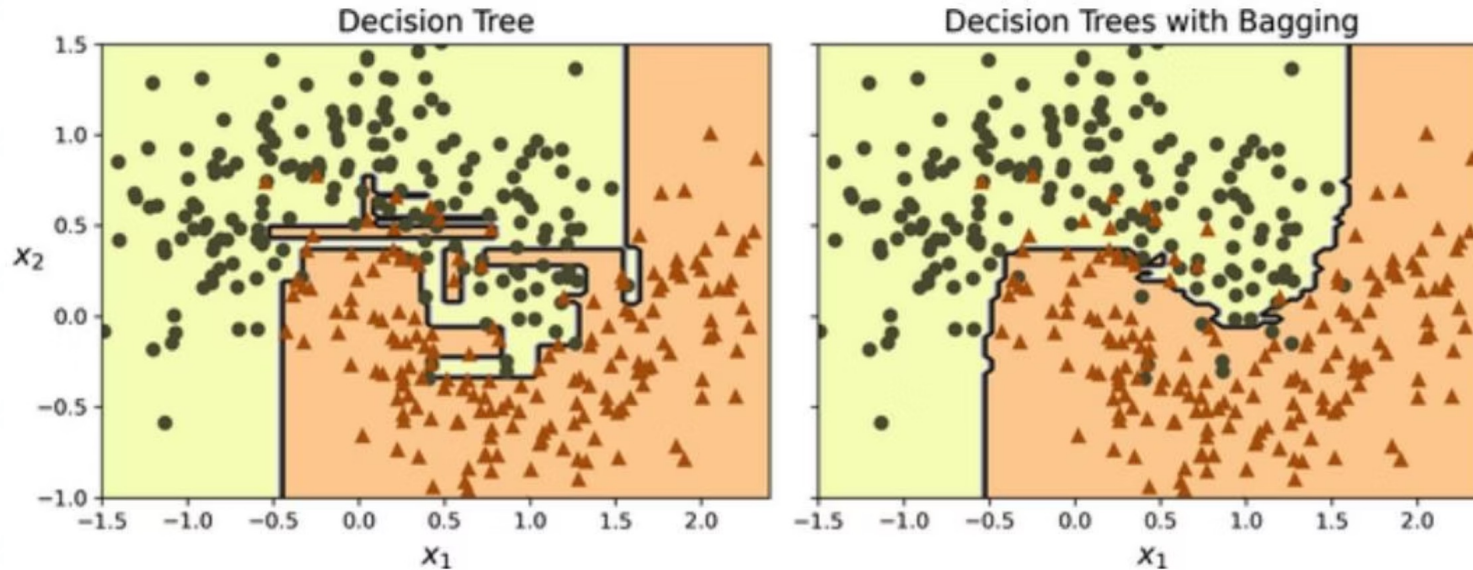
편향-분산 트레이드오프

각 개별 예측기는 원래 훈련 세트에서 훈련된 경우보다 더 높은 편향을 가지지만, 집계는 편향과 분산을 모두 감소시킴.

병렬 처리

예측기들은 모두 병렬로 훈련될 수 있어 CPU 코어나 서버를 효율적으로 활용할 수 있음. 이는 배깅과 페이스팅이 인기 있는 이유 중 하나임.

랜덤 포레스트



랜덤 포레스트는 결정 트리의 앙상블로, 일반적으로 배깅 방법을 통해 훈련됨.

랜덤 포레스트는 트리를 성장시킬 때 전체 특성을 모두 사용하는 대신 특성의 무작위 부분 집합 중에서 최상의 특성을 찾음.



다양성 증가

이 알고리즘은 더 큰 트리 다양성을 초래하여 더 높은 편향과 더 낮은 분산을 교환함. 일반적으로 전체적으로 더 나은 모델을 생성함.



특성 중요도

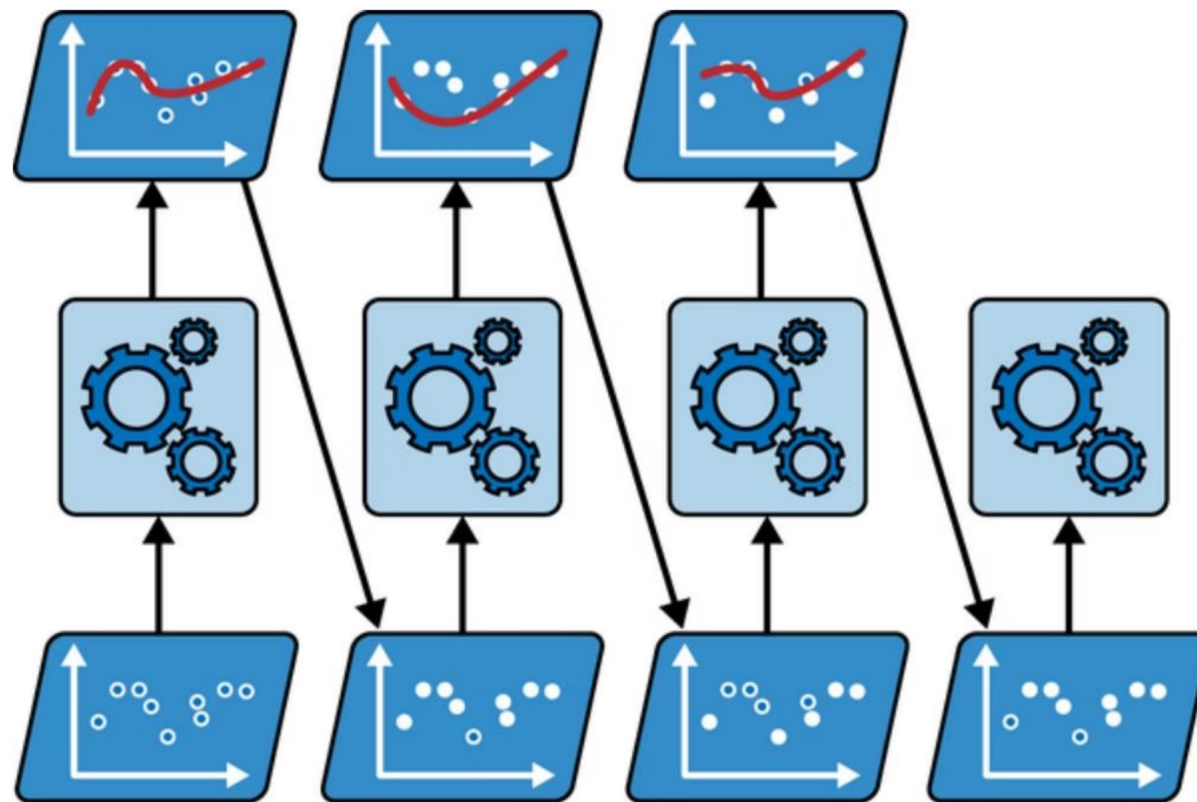
랜덤 포레스트는 각 특성의 상대적 중요도를 쉽게 측정할 수 있게 함. 이는 특성이 실제로 중요한지 빠르게 이해하는 데 매우 유용함.



엑스트라 트리

극도로 무작위화된 트리의 앙상블인 엑스트라 트리는 더 많은 무작위성을 도입하여 편향과 분산의 균형을 조정함. 훈련 속도가 더 빠른 장점이 있음.

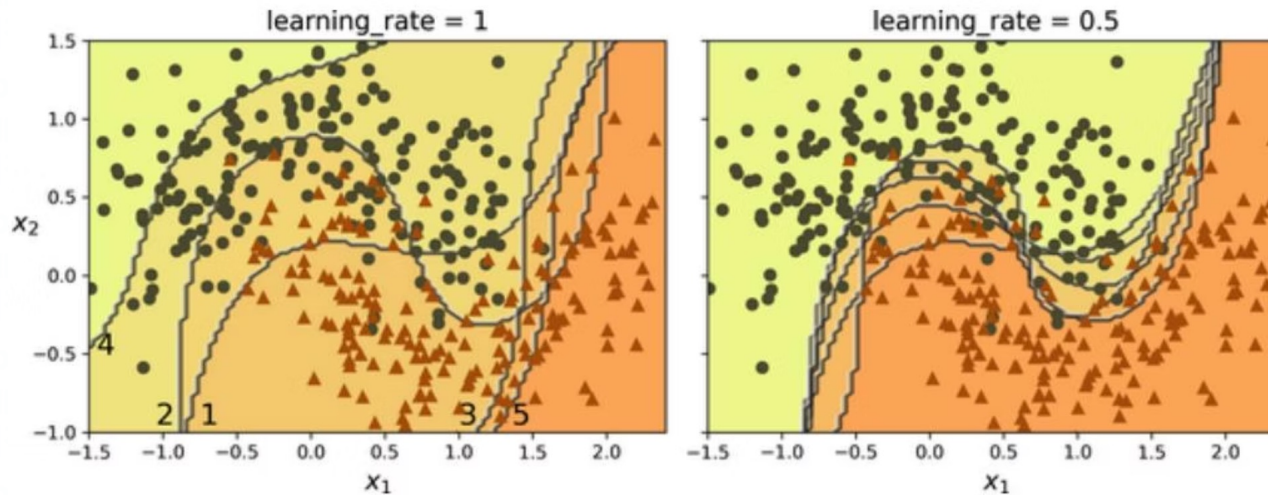
부스팅



부스팅(원래 가설 부스팅이라 불림)은 여러 약한 학습기를 강한 학습기로 결합할 수 있는 앙상블 방법을 의미함. 대부분의 부스팅 방법의 일반적인 아이디어는 예측기를 순차적으로 훈련하여 각각 이전 예측기의 오류를 수정하는 것임.

가장 인기 있는 부스팅 방법은 AdaBoost(적응형 부스팅)와 그래디언트 부스팅임.

AdaBoost



새로운 예측기가 이전 예측기가 잘 맞추지 못한 훈련 인스턴스에 더 많은 관심을 기울이는 방식. 이로 인해 새로운 예측기는 점점 더 어려운 사례에 집중.

기본 분류기 훈련

알고리즘은 먼저 기본 분류기(예: 결정 트리)를 훈련하고 이를 사용하여 훈련 세트에 대한 예측을 수행함.

순차적 학습

업데이트된 가중치를 사용하여 두 번째 분류기를 훈련하고, 다시 훈련 세트에 대한 예측을 수행하고, 인스턴스 가중치를 업데이트하는 과정을 반복.

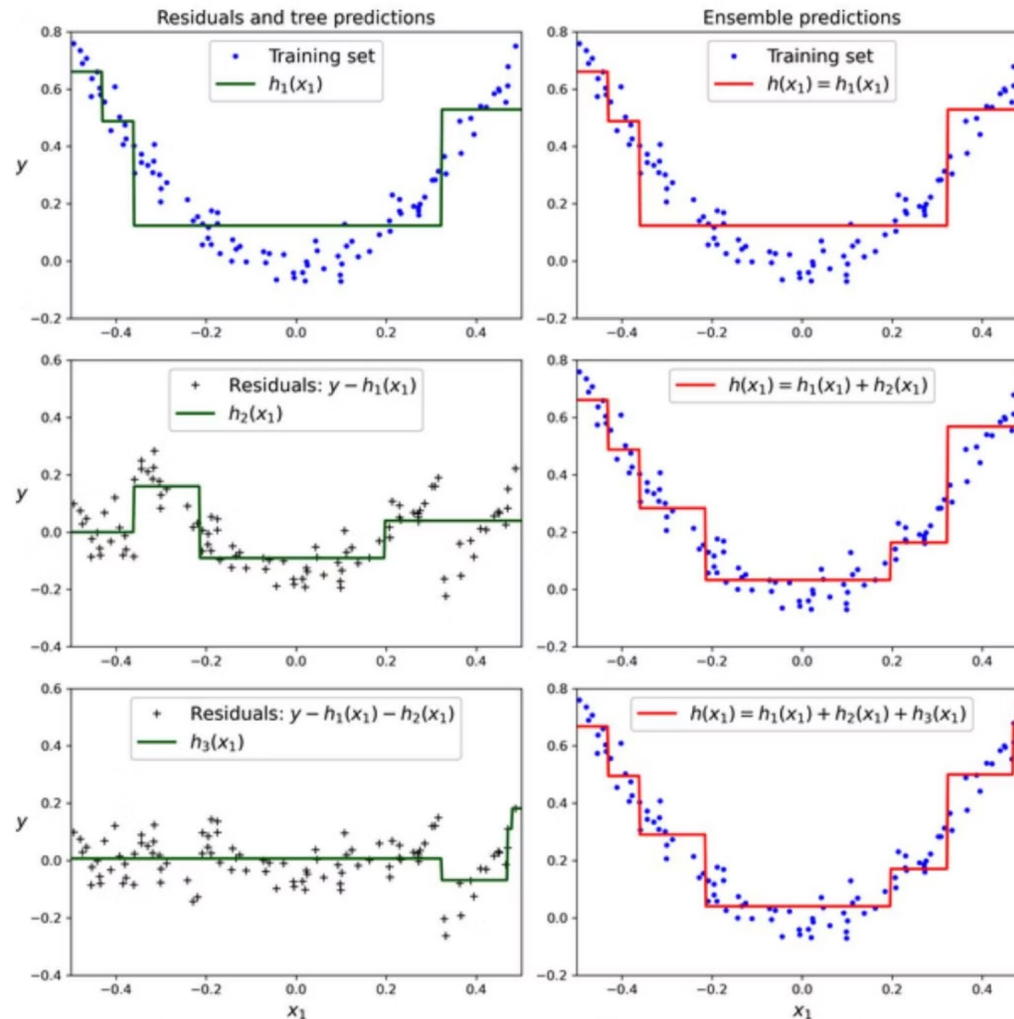
가중치 업데이트

오분류된 훈련 인스턴스의 상대적 가중치를 증가시킴. 이렇게 하면 다음 분류기가 이러한 어려운 사례에 더 집중하게 됨.

가중 투표

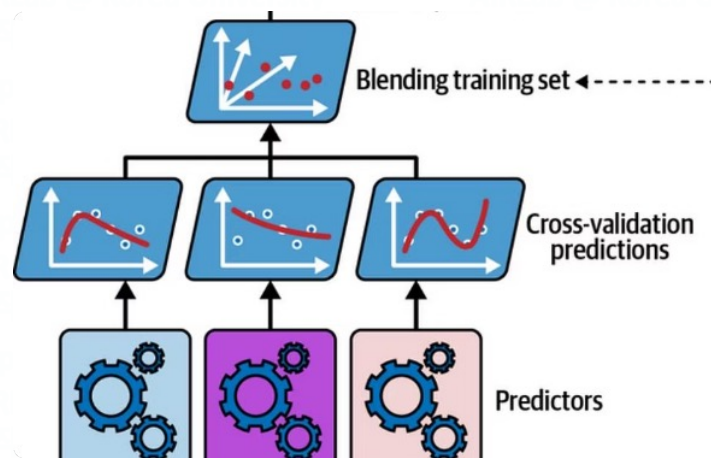
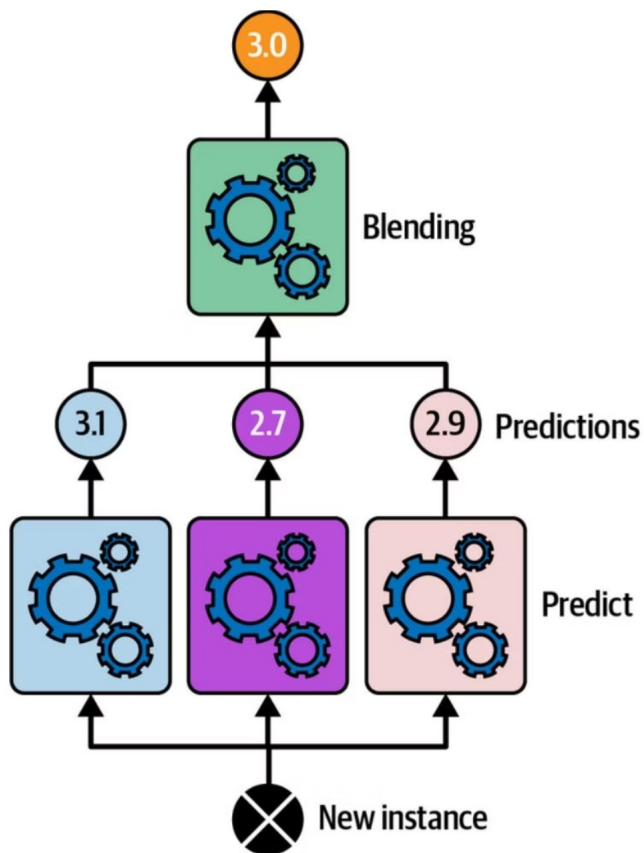
모든 예측기가 훈련되면, 앙상블은 가중치가 부여된 투표를 통해 예측을 수행함. 예측기의 가중치는 가중치가 부여된 훈련 세트에서의 전체 정확도에 따라 결정됨.

그래디언트 부스팅



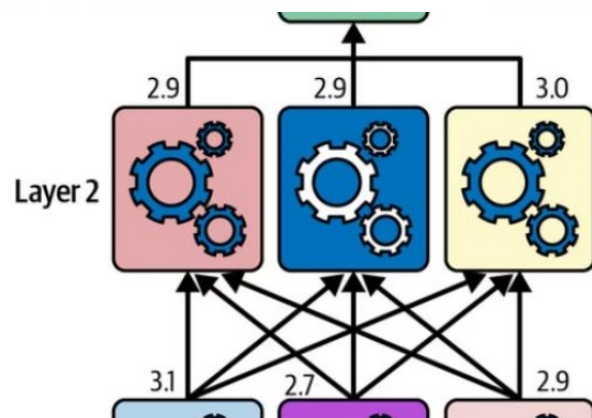
- 그래디언트 부스팅은 AdaBoost와 마찬가지로 순차적으로 예측기를 앙상블에 추가하여 각각 이전 예측기의 오류를 수정함.
- 그러나 AdaBoost처럼 인스턴스 가중치를 조정하는 대신, 이전 예측기가 만든 잔차 오류에 새로운 예측기를 맞추려고 시도.

스태킹



블렌더 훈련

블렌더(또는 메타 학습기)를 훈련시키기 위해 먼저 블렌딩 훈련 세트를 구축해야 함.



다층 스태킹

여러 다른 블렌더를 훈련시켜 블렌더 층을 형성하고, 그 위에 또 다른 블렌더를 추가하여 최종 예측을 생성할 수도 있음.

이는 성능을 약간 더 향상시킬 수 있지만, 훈련 시간과 시스템 복잡성이 증가함.

스태킹: 앙상블의 모든 예측기의 예측을 집계하기 위해 하드 보팅과 같은 단순한 함수를 사용하는 대신, 이 집계를 수행하는 모델을 훈련시킴.

앙상블 방법의 결론

앙상블 방법은 다재다능하고 강력하며 사용하기 상대적으로 간단함. 랜덤 포레스트, AdaBoost, GBRT는 대부분의 머신러닝 작업에서 가장 먼저 테스트해야 할 모델 중 하나이며, 특히 이질적인 표 형식 데이터에서 뛰어난 성능을 보임.

1 전처리 최소화

앙상블 방법은 매우 적은 전처리만 필요하므로 프로토타입을 빠르게 구축하고 실행하는 데 탁월함.

2 성능 향상

투표 분류기와 스택킹 분류기와 같은 앙상블 방법은 시스템의 성능을 극한까지 끌어올리는 데 도움이 될 수 있음.

3 다양성의 중요성

앙상블의 성능을 최대화하려면 예측기 간의 다양성을 확보하는 것이 중요함. 이는 서로 다른 알고리즘을 사용하거나 데이터의 다른 부분 집합으로 훈련함으로써 달성할 수 있음.

Thank You